Number-theoretic algorithms

Build gcd, modular equations, exponentiation, primality testing, and RSA from divisibility and invariants.

Divisibility and congruence

For $n>0$, $a \equiv b \pmod{n}$ means n divides $a-b$. For example $29 \equiv 5 \pmod{12}$, since $29-5=2 \cdot 12$. Reduce sums and products term by term.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euclid’s algorithm

For gcd(252,198): $252=1 \cdot 198+54$; $198=3 \cdot 54+36$; $54=1 \cdot 36+18$; $36=2 \cdot 18+0$. The gcd is 18.

$$252 = 1 \times 198 + 54$$

$$198 = 3 \times 54 + 36$$

$$54 = 1 \times 36 + 18$$

$$36 = 2 \times 18 + 0$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The gcd invariant

If a=qb+r, the common divisors of a,b are exactly those of b,r: r=a−qb and a=qb+r. For inputs not both zero, normalized to absolute values, the last nonzero remainder is the gcd.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extended Euclid table

Maintain r=252s+198t. Rows (r,s,t): (252,1,0), (198,0,1), (54,1,−1), (36,−3,4), (18,4,−5), (0,−11,14). Thus 18=4 • 252−5 • 198.

- $252 = 1\times252 + 0\times198$
- $198 = 0\times252 + 1\times198$
- $54 = 1\times252 - 1\times198$
- $36 = -3\times252 + 4\times198$
- $18 = 4\times252 - 5\times198$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Coefficient update and termination

From rows (r_0,s_0,t_0), (r_1,s_1,t_1), set $q=\lfloor r_0/r_1 \rfloor$. The next row is $(r_0-qr_1,s_0-qs_1,t_0-qt_1)$. Use this update only while $r_1>0$. Then $0 \le r_0-qr_1<r_1$; the linear combination persists and the positive remainders decrease.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Integer inputs and Bézout coefficients

For $(a,b)=(0,0)$, define gcd=0 and choose $x=y=0$. Otherwise start with $r_0=|a|$, $r_1=|b|$ and coefficient rows $(1,0)$, $(0,1)$ for $|a|$ and $|b|$. Stop when $r_1=0$, before division. The last nonzero r_0 is gcd(a,b).

If $r_0=s|a|+t|b|$, return $x=s \cdot \operatorname{sign}(a)$, $y=t \cdot \operatorname{sign}(b)$, with $\operatorname{sign}(0)=0$. Then $ax+by=\gcd(a,b)$, including either zero input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Signed input examples

For $(7,-3)$, use $(7,3)$: $7=2\cdot 3+1$; $3=3\cdot 1+0$. Back substitute $1=7-2\cdot 3=1\cdot 7+2\cdot(-3)$, so $(x,y)=(1,2)$.

For $(-12,18)$, use $(12,18)$: $12=0\cdot 18+12$; $18=1\cdot 12+6$; $12=2\cdot 6+0$. Back substitute $6=18-12=1\cdot(-12)+1\cdot 18$, so $(x,y)=(1,1)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zero input examples

For $(0,-5)$, use $(0,5)$: $0=0\cdot 5+0$, then stop. The gcd is $5=0\cdot 0+(-1)\cdot(-5)$, so $(x,y)=(0,-1)$.

For $(0,0)$, no division occurs. By convention $\gcd(0,0)=0=0\cdot 0+0\cdot 0$, with $(x,y)=(0,0)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euclid: worst-case divisions

For $a \geq b > 0$, suppose Euclid produces k nonzero remainders. Reverse the recurrence: the smallest possible inputs are consecutive Fibonacci numbers, so $b \geq F_{k+1}$ and $F_j \geq \phi^{j-2}$ for $j \geq 2$, where $\phi = (1+\sqrt{5})/2$. Thus there are $O(\log(b+1))$ divisions, including $b=1$. As b grows, consecutive Fibonacci inputs attain $\Theta(\log b)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why Fibonacci numbers appear

Let $g = \gcd(a,b) > 0$ and divide every remainder by g. The final normalized remainders are 1,0; the unnormalized final nonzero remainder is g. Working backward through the normalized sequence, $r_{i-1} = q_i r_i + r_{i+1}$ with $q_i \geq 1$, so $r_{i-1} \geq r_i + r_{i+1}$. Taking quotient 1 yields the Fibonacci lower bound. For example, $34 = 21+13$, $21 = 13+8$, $13 = 8+5$, $8 = 5+3$, $5 = 3+2$, $3 = 2+1$, $2 = 2 \cdot 1$.

Reconstruct the smallest remainders

Each value is at least the sum of the next two

34 ▶ 21 ▶ 13 ▶ 8 ▶ 5 ▶ 3 ▶ 2 ▶ 1

Equality occurs when the quotient is 1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Division count versus bit cost

If both inputs have at most B bits, classical long division costs $O(B^2)$ bit operations per division. Combining that with O(B) divisions yields the simple $O(B^3)$ upper bound. This bound is deliberately conservative, not a claim of optimal implementation; O(B) arithmetic divisions is not the same as O(B) bit operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euclid halves within two divisions

For $a \geq b > 0$ let $r = a \bmod b$. If $r \leq b/2$, the second argument has at least halved in one division. If $r > b/2$, then $b/2 < r < b$, so the next quotient is 1 and $b \bmod r = b - r < b/2$. Thus within at most two divisions the new second argument is at most half the old one, or the algorithm terminates. This gives O(log(b+1)) divisions without treating large-integer division as constant bit cost.

For example $391 = 299 + 92$, $299 = 3 \cdot 92 + 23$ and $92 = 4 \cdot 23 + 0$, giving gcd=23. Back-substitution yields $23 = 299 - 3(391 - 299) = 4 \cdot 299 - 3 \cdot 391$. Because $\gcd(299, 391) = 23 \neq 1$, 299 has no inverse modulo 391: an inverse would imply $299u - 391v = 1$ although the left side is divisible by 23.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Solvability of linear congruences

ax≡b (mod n) iff there is an integer y with ax+ny=b. Let d=gcd(a,n). For n>0, a solution exists iff d|b, and when it does there are exactly d distinct residues modulo n: necessity follows from divisibility; sufficiency follows by multiplying Bézout’ s identity by b/d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

All solutions of a congruence

Solve 14x≡30 (mod 100). Divide by d=2: 7x≡15 (mod 50). Since $7^{-1}\equiv 43$, x≡45 (mod 50). Exactly two residues modulo 100, 45 and 95, both yield remainder 30.

- 14x ≡ 30 (mod 100)
- 7x ≡ 15 (mod 50)
- x ≡ 45 (mod 50)
- x = 45, 95 (mod 100)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Modular inverses

a has an inverse modulo n iff gcd(a,n)=1. For example $7 \cdot 43 = 301 \equiv 1 \pmod{50}$. Extended Euclid supplies the Bézout coefficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linear congruences: compact solution sets

Solve $ax \equiv b \pmod{n}$, $n \geq 1$. Let $g = \gcd(a,n)$. If g does not divide b there is no solution. If g divides b, divide the equation and modulus by g. When $\text{step} = n/g > 1$, a/g is coprime to step and its inverse gives one x_0. When step=1, take $x_0 = 0$ directly without invoking an inverse modulo 1. The complete set modulo n is $x_0 + j \cdot \text{step}$ for $0 \leq j < g$, where $\text{step} = n/g$ and $0 \leq x_0 < \text{step}$. The tuple (x_0, step, g) encodes the full set.

Example: $24x \equiv 36 \pmod{60}$ has $g = 12$ and reduces to $2x \equiv 3 \pmod{5}$. Thus $x_0 = 4$, step=5, and (4,5,12) represents the 12 solutions 4,9,…,59. Every residue solves $0x \equiv 0 \pmod{n}$, represented by (0,1,n), including n=1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compact output versus enumeration

After reducing a,b modulo n, each tuple component is at most n, so the representation uses $O(\log(n+1))$ bits. That is output size alone, not the computation cost for gcd or an inverse.

Listing all g solutions requires at least $\Omega(g)$ output operations. For $0x \equiv 0 \pmod{n}$, $g=n$; writing all representatives in ordinary binary takes $\Theta(n \log(n+1))$ output bits. Given n in $\beta=\lceil \log_2(n+1) \rceil$ bits, n can grow exponentially in β. A compact representation and explicit enumeration therefore have different costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Modular division means multiplying an inverse

Only residues coprime to n have multiplicative inverses modulo n. If $b \in U(n)$, a/b denotes $a \cdot b^{-1} \pmod{n}$, not integer division followed by reduction. For example $7 \cdot 13 = 91 \equiv 1 \pmod{15}$, so $4/7 \equiv 4 \cdot 13 \equiv 7 \pmod{15}$. A non-unit denominator cannot be cancelled directly; solve the linear congruence and check its gcd condition instead.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chinese remainder theorem

For pairwise coprime moduli the prescribed residues have one solution modulo their product. For $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, candidates 2,5,8 show $x \equiv 8 \pmod{15}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Non-coprime compatibility

$x \equiv a \pmod m$, $x \equiv b \pmod n$ is solvable iff $\gcd(m,n) \mid (b-a)$. The solution is unique modulo $\operatorname{lcm}(m,n)$. Substitute $x = a + mt$ to obtain $mt \equiv b - a \pmod n$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Non-coprime example

For $x \equiv 2 \pmod 6$, $x \equiv 5 \pmod 9$, gcd=3 divides the difference 3. Solve $6t \equiv 3 \pmod 9$, obtaining $t \equiv 2 \pmod 3$, hence $x \equiv 14 \pmod{18}$. Replacing 5 by 4 makes the system impossible.

$$x = 2 + 6t$$

$$6t \equiv 3 \pmod 9$$

$$t \equiv 2 \pmod 3$$

$$x \equiv 14 \pmod{18}$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CRT construction for coprime moduli

For pairwise coprime $m_1,\ldots,m_k>1$ and arbitrary residues a_i, set $M=\prod m_i$ and $M_i=M/m_i$. Choose $y_i=M_i^{-1} \bmod m_i$; then $x \equiv \Sigma a_i M_i y_i \pmod M$.

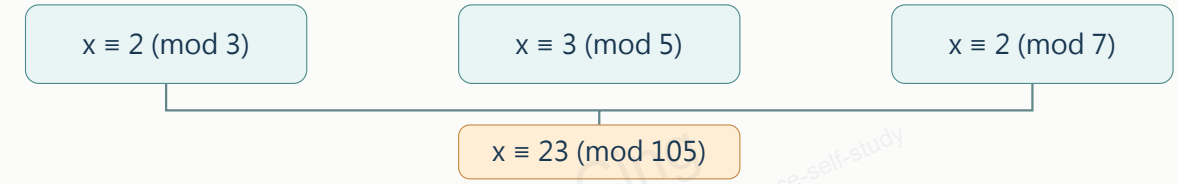
Modulo m_i, only the i-th term survives, and it equals a_i. If x and x′ are solutions, every m_i divides $x-x'$, so pairwise coprimality implies M divides the difference. The solution is unique modulo M.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CRT example

For $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$, $M = 105$ and $(M_1, M_2, M_3) = (35, 21, 15)$. Their respective inverses are $(2, 1, 1)$, so $x \equiv 2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 2 + 2 \cdot 15 = 233 \equiv 23 \pmod{105}$. Check the three remainders: 2, 3, 2.

One integer satisfies all three constraints



23 has remainders 2, 3, 2 modulo 3, 5, 7

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CRT for non-coprime moduli

The congruences $x \equiv a \pmod m$, $x \equiv b \pmod n$ are solvable exactly when $d = \gcd(m, n)$ divides $b - a$. Necessity follows by subtracting the congruences. For sufficiency put $x = a + mt$ and solve $(m/d)t \equiv (b - a)/d \pmod{n/d}$. Since m/d and n/d are coprime, t is unique modulo n/d and x is unique modulo $\operatorname{lcm}(m, n)$. Merge several constraints successively; stop if a pair is incompatible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Modular exponentiation

For $m>1$, $e\geq 0$, initialize accumulator=1 and scan all bits of e from most to least significant. Square acc modulo m, then multiply by a modulo m if the bit is 1. For e=0 the result is 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bit-by-bit exponentiation

Compute $7^{13} \bmod 23$, with $13=(1101)_2$. Starting from 1, the first bit gives $1^2 \cdot 7 \bmod 23=7$. The second gives $7^2 \bmod 23=3$, then $3 \cdot 7=21$. After each bit the accumulator is 7, 21, 4, 20. The answer is 20.

bits: 1 1 0 1

acc: 1 → 7 → 21 → 4 → 20

$7^{13} \bmod 23 = 20$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exponentiation invariant

After a prefix representing k, acc$\equiv a^k \pmod m$. Appending bit b forms 2k+b; squaring and multiplying by a^b establishes the next row. O(log(e+1)) counts modular multiplications, not bit operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Right-to-left modular exponentiation

Inputs are $A \in \mathbb{Z}$, $B \geq 0$, $n \geq 1$. Initialize r=1 mod n, a=A mod n and b=B. While b>0, if b is odd set $r \leftarrow ra \bmod n$, then set $a \leftarrow a^2 \bmod n$ and $b \leftarrow \lfloor b/2 \rfloor$. The invariant is $r \cdot a^b \equiv A^B \pmod n$.

For b=2q, the updated product is $r(a^2)^q = r \cdot a^b$. For b=2q+1 it is $(ra)(a^2)^q = r \cdot a^b$. Each round halves b; at b=0 the invariant yields $r \equiv A^B$. If B=0 return 1 mod n immediately; in particular n=1 yields 0. Non-coprime bases work because no inverse is used. Negative exponents are outside this algorithm; they require a separately defined inverse.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Right-to-left modular exponentiation trace

For $7^{19} \bmod 23$ start at (r,a,b)=(1,7,19). Each row preserves the invariant; the final result is 11.

Before (r,a,b)	After (r,a,b)
(1,7,19)	(7,3,9)
(7,3,9)	(21,9,4)
(21,9,4)	(21,12,2)
(21,12,2)	(21,6,1)
(21,6,1)	(11,13,0)

For example, in row 3, b=4 is even: r remains 21, a becomes $9^2 \equiv 12$ and b becomes 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finite closed subsets and element order

A nonempty subset S of a finite group G that is closed under the group operation is a subgroup. For $a \in S$, all $a, a^2, \ldots$ lie in S; finiteness gives $a^i = a^j$ for some $1 \le i < j$. Cancellation in G gives $a^{j-i} = e \in S$. If $j - i > 1$, $a^{j-i-1} \in S$ is the inverse of a; if $j - i = 1$ then $a = e$, whose inverse is also in S. Finiteness is essential.

The order of a is the least positive t with $a^t = e$. The elements $e, a, \ldots, a^{t-1}$ are distinct and all powers repeat modulo t, so $|\langle a \rangle| = t$. Lagrange gives $t \mid |G|$ and hence $a^{|G|} = e$. In additive Z_8, 2 generates {0,2,4,6} and has order 4; in multiplicative U(7), 2 generates {1,2,4} and has order 3. The operation determines the meaning of $\langle 2 \rangle$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lagrange’ s theorem: partition into cosets

Let G be a finite group and H a subgroup. A left coset is $aH=\{ah:h\in H\}$. The map $h\mapsto ah$ is a bijection: left multiplication by a^{-1} recovers h. Thus every coset has $|H|$ elements.

Two cosets are either identical or disjoint: if $ah_1=bh_2$, then $b^{-1}a=h_2h_1^{-1}\in H$ and consequently $aH=bH$. Every a lies in aH because H contains the identity. Thus the cosets partition G into equal-sized blocks and $|G|=[G:H]|H|$. In particular $|H|$ divides $|G|$. For $H=\langle g\rangle$, $\mathrm{ord}(g)$ divides $|G|$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euler ϕ: prime powers and multiplicativity

For prime p and integer $e\geq 1$, precisely p^{e-1} of $1,\ldots,p^e$ are divisible by p; all other entries are coprime to p^e. Therefore $\phi(p^e)=p^e-p^{e-1}=p^{e-1}(p-1)$.

For $\gcd(m,n)=1$, CRT gives a bijection of residue classes. A residue is a unit modulo mn exactly when its components are units modulo m and n. Therefore $U(mn)\cong U(m)\times U(n)$, and counting gives $\phi(mn)=\phi(m)\phi(n)$. Factor a general modulus into pairwise-coprime prime powers and multiply. Example: $72=8\cdot 9$, so $\phi(72)=\phi(8)\phi(9)=4\cdot 6=24$; set $\phi(1)=1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finite groups, Euler’ s theorem, and discrete logs

The invertible residues U(n) form a multiplicative group of size ϕ(n). Multiplication by a∈U(n) permutes U(n); multiplying all residues and cancelling invertible factors gives $a^{\varphi(n)} \equiv 1 \pmod{n}$. More precisely, the order $\mathrm{ord}_n(a)$ divides ϕ(n), since its generated subgroup has order dividing the group order.

If $\mathrm{ord}_n(g)=\phi(n)$, g is a primitive root and U(n)=⟨g⟩. For h∈⟨g⟩ a discrete logarithm is x satisfying $g^x=h$, unique modulo ord(g); outside the subgroup no solution exists. Not every modulus has a primitive root.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Group and discrete-log example

In U(7)={1,2,3,4,5,6}, successive powers of 3 are 3,2,6,4,5,1. Thus $\mathrm{ord}_7(3)=6$, 3 is primitive, and $\log_3 5 \equiv 5 \pmod 6$. Powers of 2 are 2,4,1, generating {1,2,4}; hence $2^x \equiv 3 \pmod 7$ has no solution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why a generator exists modulo a prime

Let G be the nonzero multiplicative group of F_p. Let L be the least common multiple of the orders of all elements of G. For each maximal prime-power factor ℓ^a of L, choose an element whose order contains that factor and take an appropriate power to obtain an element of order exactly ℓ^a.

These prime-power orders are pairwise coprime. Since G is abelian, their product has order L: if its k-th power is 1, each factor's k-th power lies both in its own subgroup and in the subgroup generated by all other factors. These subgroup orders are coprime, so their intersection is the identity and each factor order divides k; conversely the L-th power is 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finish using the polynomial root bound

Every $x \in G$ satisfies $x^L=1$. A nonzero polynomial X^L-1 over a field has at most L roots, so $p-1 \le L$. Lagrange's theorem says every element order divides $p-1$, hence their least common multiple L divides $p-1$ and $L \le p-1$. Thus $L=p-1$ and the element constructed above generates G.

The root bound depends on working over a field; it cannot be applied unchanged to an arbitrary composite-modulus residue ring.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lift a generator from p to p^2

Let p be an odd prime and g a primitive root modulo p. Compare g and g+p. The binomial theorem gives $(g+p)^{p-1} \equiv g^{p-1} + (p-1)g^{p-2}p \pmod{p^2}$. The second term is a nonzero multiple of p modulo p^2; therefore at least one choice h satisfies $h^{p-1} \not\equiv 1 \pmod{p^2}$. As $h^{p-1} \equiv 1 \pmod{p}$, write it as 1+cp with p∤c.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lift to every odd prime power

For odd p and $v_p(z-1)=j \geq 1$, expand $z^p-1=(1+(z-1))^p-1$. The leading term has p-adic valuation j+1 and every other term has greater valuation. Hence $v_p(z^p-1)=j+1$. Induction gives $v_p(h^{(p-1)p^k}-1) = k+1$.

The order modulo p^e is a multiple of p−1 and divides $\phi(p^e)=(p-1)p^{e-1}$. The valuation excludes every smaller power of p, so the order is $(p-1)p^{e-1}$ and $U(p^e)$ is cyclic. For p=3, g=2 gives $2^2=4 \not\equiv 1 \pmod{9}$, so 2 generates $U(3^e)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Which moduli have primitive roots?

For $n>1$, $U(n)$ is cyclic exactly for $n=2$, 4, p^e or $2p^e$, where p is an odd prime and $e\geq 1$. $U(2)$ and $U(4)$ can be listed directly; $U(p^e)$ was proved cyclic above. CRT gives $U(2p^e)\cong U(2)\times U(p^e)$, and the first component has one element.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exclude the remaining moduli

If $8|n$, write $n=2^a m$ with $a\geq 3$ and m odd. For any $b\in U(8)$, choose its odd integer representative as a unit modulo 2^a. When $m>1$, apply CRT to the coprime moduli 2^a and m to obtain $x\equiv b \pmod{2^a}$, $x\equiv 1 \pmod{m}$; when $m=1$, take $x=b$. Then x is a unit modulo n and $x\equiv b \pmod 8$, proving the natural projection $U(n)\to U(8)$ is surjective. Every element of the four-element group $U(8)=\{1,3,5,7\}$ squares to 1, so it is not cyclic. A homomorphic image of a cyclic group is cyclic, a contradiction.

If n has two distinct odd prime factors, CRT allows independent choices of ± 1 and yields at least four unit solutions of $x^2=1$. The same holds if $4|n$ and n has an odd prime factor, using $U(4)$ and the odd component. A finite cyclic group has at most two solutions to $x^2=1$. These exclusions cover every other n.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A primitive root modulo 121

The order of 2 modulo 11 is 10. To lift it to $11^2=121$, check $2^{10}=1024\equiv 56\not\equiv 1 \pmod{121}$. Its order modulo 121 is a multiple of its order 10 modulo 11 and divides $\phi(121)=110$, so it is either 10 or 110. The first case is excluded, giving order 110: 2 generates $U(121)$.

Only after the non-1 check modulo p^2 can the valuation argument above lift the same representative to higher odd prime powers. A generator modulo p does not automatically make every chosen integer representative a generator modulo p^2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The only square roots of 1 modulo an odd prime power

Let p be an odd prime and $e\geq 1$. If $x^2\equiv 1 \pmod{p^e}$, then $p^e \mid (x-1)(x+1)$. As $\gcd(x-1,x+1)$ divides 2, p cannot divide both factors. Thus all e copies of p divide one side and $x\equiv 1$ or $-1 \pmod{p^e}$. Both residues are roots and they are distinct because p is odd.

For example modulo 25 the only roots are 1 and 24. A composite modulus can have more: 1,4,11,14 are all roots modulo 15, arising from independent sign choices in the CRT components.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A nontrivial square root exposes factors

Let odd $n>1$ and $x^2 \equiv 1 \pmod{n}$, with $x \not\equiv 1, -1 \pmod{n}$. Set $d_- = \gcd(x-1, n)$, $d_+ = \gcd(x+1, n)$. Both are less than n, otherwise $x \equiv \pm 1$. If $d_- = 1$, cancellation in $n \mid (x-1)(x+1)$ forces $n \mid (x+1)$, a contradiction; similarly $d_+ \neq 1$. Both are nontrivial factors.

For $n=77$ and $x=34$, $34^2 = 1156 \equiv 1 \pmod{77}$, while $\gcd(33,77)=11$ and $\gcd(35,77)=7$. In CRT coordinates, $34 \equiv 1 \pmod{11}$ and $34 \equiv -1 \pmod{7}$: the components chose different signs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Primes and composites

Trial division up to $\sqrt{n}$ decides $n \geq 2$ but can take many steps for long inputs. For example $91 = 7 \cdot 13$ is composite; For 97, test the primes 2, 3, 5, and 7 up to $\sqrt{97}$; none divides it, so 97 is prime.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Miller–Rabin preconditions

For $n<2$ return not prime; 2 and 3 are prime; even $n>2$ is composite. For odd $n\geq 5$ write $n-1=2^s d$ with d odd, and choose $a\in\{2,\ldots,n-2\}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Witness test

Compute $x=a^d \bmod n$. Pass if $x=1$ or $n-1$; otherwise square at most $s-1$ times and pass if $n-1$ appears. Otherwise a witnesses compositeness. For $n=15$, $a=2$, $d=7$, $s=1$, $x=8$, so the test fails.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why primes always pass

For odd prime p, $x^2 \equiv 1 \pmod p$ implies $x \equiv \pm 1$ because p divides (x−1)(x+1). Together with Fermat’ s theorem $a^{p-1} \equiv 1$, this forces every legal base to pass.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Random error bound and scope

For a fixed odd composite $n \geq 5$, choose each base independently and uniformly from 2… n−2. The probability that all t rounds pass is at most 4^{-t}. This is conditional on a fixed composite, not a posterior probability that an arbitrary input is composite.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fermat pseudoprime: 341

$341 = 11 \cdot 31$ is composite, yet $2^{10} = 1024 \equiv 1 \pmod{341}$, hence $2^{340} = (2^{10})^{34} \equiv 1$. A base-2 Fermat test accepts 341; that does not make it prime.

For Miller–Rabin, $340 = 2^2 \cdot 85$. We have $2^{85} = (2^{10})^8 \cdot 2^5 \equiv 32$; the next square is $32^2 = 1024 \equiv 1 \pmod{341}$. The initial value is neither 1 nor 340, and -1 did not appear before reaching 1, so the test rejects 341. Here 32 is a nontrivial square root of 1: $\gcd(32-1, 341) = 31$ and $\gcd(32+1, 341) = 11$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Carmichael number: 561

$561 = 3 \cdot 11 \cdot 17$ is composite, and 560 is a multiple of $3-1=2$, $11-1=10$ and $17-1=16$. For $\gcd(a, 561) = 1$, Fermat's theorem gives $a^{560} \equiv 1$ modulo each prime factor, so CRT gives $a^{560} \equiv 1 \pmod{561}$. Every base coprime to 561 passes its Fermat test; the claim does not apply to non-coprime bases.

For Miller–Rabin, $560 = 2^4 \cdot 35$. Base 2 yields $2^{35} \equiv 263$ followed by squares 166,67,1 modulo 561. The initial value is not 1 and none of the checked values is 560, so base 2 exposes compositeness. All coprime bases fool Fermat here; they do not all fool the strong test.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strong pseudoprime: 2047

$2047=23\cdot 89$ is composite, and $2046=2\cdot 1023$, so $s=1$ and $d=1023$. Since $2^{11}=2048\equiv 1 \pmod{2047}$ and $1023=11\cdot 93$, $2^{1023}\equiv 1$. The base-2 Miller–Rabin round therefore accepts 2047; 2 is a strong liar for this composite.

The comparison concerns one round with a fixed base. The randomized Miller–Rabin error bound requires independently and uniformly sampling bases as specified; always using base 2 does not provide that probability guarantee.

Base 2: results for three composite numbers

Number	Fermat	Miller–Rabin	Evidence
$341 = 11\cdot 31$	passes	rejects	$32 \to 1$
$561 = 3\cdot 11\cdot 17$	passes	rejects	$263 \to 166 \to 67 \to 1$
$2047 = 23\cdot 89$	passes	passes	$2^{1023} \equiv 1$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exact strong liars for 65

$65=5\cdot 13$ and $64=2^6\cdot 1$, so $s=6$ and $d=1$. Since $\phi(5)=2^2\cdot 1$ and $\phi(13)=2^2\cdot 3$, both components have $\gcd(1,u_i)=1$ and $t_i=2$. The count is $L=1+1+4=6$.

Initial value 1 gives $a=1$; initial value -1 gives $a=64$. The next layer requires $a^2\equiv -1 \pmod{65}$. Its roots modulo 5 are 2,3 and modulo 13 are 5,8; CRT gives 8,18,47,57. The full liar set is $\{1,8,18,47,57,64\}$; sampling only from $2,\ldots,63$ retains four liars, with acceptance probability $4/62=2/31$. Base 2 has the six checked values 2,4,16,61,16,61: its initial value is not 1 and none equals 64, so it is a witness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Miller–Rabin: setting up the 1/4 bound

Let odd composite $n = \prod_i p_i^{e_i}$ and n−1=2ˢd with d odd. Write $\varphi(p_i^{e_i}) = 2^{t_i} u_i$ with u_i odd, set A=∏gcd(d,u_i) and t=min(s,t_1,…,t_k). The multiplicative group modulo an odd prime power is cyclic. In a cyclic group of order $2^{t_i}u_i$, $z^d = 1$ has gcd(d,u_i) solutions.

Also, $z^{2^r d} = -1$ has 2^rgcd(d,u_i) solutions for $0 \le r < t_i$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exact count of strong liars

A non-unit is necessarily a witness. CRT separates a unit into its prime-power coordinates. The strong test accepts exactly when all d-th powers are 1, or there is one common r∈[0,s) such that every 2rd-th power is −1. These cases are disjoint, giving $L = A\left(1 + \sum_{r=0}^{t-1} 2^{kr}\right)$, where k is the number of distinct prime factors. This is a count, not yet a probability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting bound: at least three prime factors

For $k \geq 3$, $\phi(n) = \prod 2^{t_i} u_i \geq 2^{kt} A$. Also $\left(1 + \sum_{r=0}^{t-1} 2^{kr}\right) / 2^{kt} \leq 1/4$: at $t=1$ this is $2/2^k \leq 1/4$; at $t>1$ the geometric sum gives a smaller ratio. Thus $L \leq \phi(n)/4 \leq (n-1)/4$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting bound: two prime factors

If the two t_i differ, $\phi(n) \geq 2^{2t+1} A$ and $1 + \sum_{r=0}^{t-1} 4^r \leq 4^t/2$, so $L \leq \phi(n)/4$. If they are equal but some $\gcd(d, u_i) < u_i$ or some $e_i > 1$, the lost odd factor is at least 3. Hence $\phi(n) \geq 3 \cdot 2^{2t} A$ and $L \leq \phi(n)/4$.

The remaining case has distinct odd primes p, q to the first power, equal t and u_1, u_2 dividing d. Since $pq-1$ is divisible by 2^t, $s \geq t$; hence $p-1$ and $q-1$ both divide $pq-1$. Reducing modulo $p-1$ and $q-1$ gives $p-1 \mid q-1$ and $q-1 \mid p-1$, forcing $p=q$, a contradiction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting bound: odd prime powers

For $n=p^e$ with $e\geq 2$, p does not divide d. Thus $A=\gcd(d, \text{the odd part of } p^{e-1}(p-1))$ is at most the odd part of $p-1$. Since $t\leq v_2(p-1)$, $L=A\cdot 2^t\leq p-1\leq (p^e-1)/4$ for odd $p\geq 3$ and $e\geq 2$.

Combining the cases gives $L\leq (n-1)/4$. A uniformly random base in $1,\ldots,n-1$ therefore has error probability at most $1/4$. The same holds for $2,\ldots,n-2$ after removing the two guaranteed liars $1,n-1$. Independent rounds give $\leq 4^{-r}$; a prime is never incorrectly rejected.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fixed-composite error versus posterior probability

For each fixed odd composite, k independent rounds with uniformly sampled legal bases all pass with probability at most $\varepsilon=4^{-k}$. To make $\varepsilon\leq 10^{-9}$, note $4^{14}=268435456<10^9\leq 4^{15}=1073741824$, so 15 is the smallest choice certified by this universal bound. This does not claim that every input needs 15 rounds to meet the target in actual error probability.

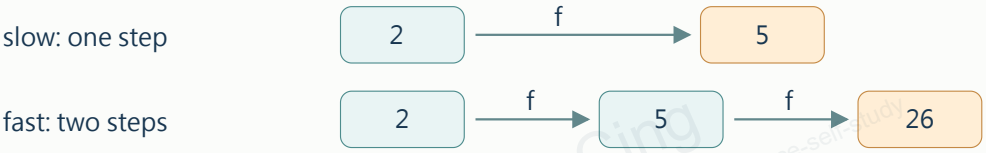
Assume the candidate distribution has prior prime probability $\pi=1/100$, primes always pass, and every composite passes with probability at most ε. Bayes' rule gives $P(\text{composite}|\text{pass})\leq (1-\pi)\varepsilon/[\pi+(1-\pi)\varepsilon]$. At $k=15$ this is $99/(1073741824+99)\approx 9.22\times 10^{-8}$. This posterior upper bound depends on the stated prior; it is not the fixed-composite false-positive probability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pollard rho and Floyd collisions

Choose $f(x)=(x^2+1) \bmod n$. Each round update slow=f(slow), fast=f(f(fast)), then $d=\gcd(|\text{slow}-\text{fast}|,n)$. A value $1<d<n$ is a nontrivial factor. If $d=n$, this run failed to separate a factor; retry with a new seed or function. If $d=1$, continue. A discovered factor need not be prime, so factor recursively or test it.

Floyd's two speeds: $n=91$, $f(x)=x^2+1 \bmod 91$



$\gcd(|5-26|,91)=7$ yields factor 7

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pollard rho example and heuristic

Let $n=91$ and $x_0=2$. In the first round slow=f(2)=5, fast=f(f(2))=f(5)=26, and $\gcd(|5-26|,91)=\gcd(21,91)=7$. Thus $91=7 \cdot 13$. This fixed seed and function happen to succeed; other runs need not.

If the sequence modulo the unknown smallest prime factor p behaves like a random mapping, the birthday-collision heuristic suggests approximately $O(\sqrt{p})$ iterations. That model is not a worst-case guarantee and excludes the bit cost of modular multiplication and gcd. A bounded run can be restarted; do not present the heuristic as a guaranteed termination theorem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three Floyd rounds for 8051

Fix n=8051, x_0=2 and f(x)=x^2+1 mod 8051. Each round advances slow once and fast twice; compute d=gcd(|slow−fast|,8051).

Round	slow	fast	Absolute difference	d
1	5	26	21	1
2	26	7474	7448	1
3	677	871	194	97

In round 3, both 677 and 871 are 95 modulo 97; modulo 83 their residues are 13 and 41. Thus gcd(194,8051)=97 and 8051=97 · 83. Trial division through each square root confirms the two factors are prime.

Third round compared modulo each factor

slow = 677		fast = 871
677 mod 97 = 95	=	871 mod 97 = 95
677 mod 83 = 13	≠	871 mod 83 = 41

Equal modulo 97, unequal modulo 83; hence gcd = 97

A different schedule: checkpoints

Define x_{i+1}=f(x_i). For j=0,1,2,… save y=x_{2^j}, and advance x through $x_{2^j+1},\ldots,x_{2^{j+1}}$, computing gcd(|x−y|,n) at every step. Compare before updating the next checkpoint. Return when 1<d<n; if d=n, retry with a new function or seed. Exhausting the work budget only means that this run did not find a factor.

A trajectory in a finite function graph eventually enters a cycle of length λ. Once y is in that cycle, an interval of at least λ subsequent steps reaches the same component value as y. The doubling schedule uses a constant number of integer states. It is not the same trace as Floyd’ s slow/fast schedule.

Checkpoint trace

Use n=8051 and x_0=2 again. The sequence is x_1=5, x_2=26, x_3=677, x_4=7474, x_5=2839, x_6=871, x_7=1848.

checkpoint y	successive values of x	gcd results
x_1=5	x_2=26	1
x_2=26	x_3=677, x_4=7474	1, 1
x_4=7474	x_5=2839, x_6=871, x_7=1848	1, 1, 97

At x_7 the gcd is 97, a proper factor of 8051, so stop. This comparison order differs from Floyd's although the underlying sequence x_i is the same.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deriving the birthday-collision bound

Model k independent uniform draws from a set of size p. For k≤p, after j distinct draws the next avoids them with probability 1−j/p. Multiplication gives P(no collision)=$\prod_{j=0}^{k-1}(1-j/p)$. Using $1-u\le e^{-u}$, P(no collision)≤exp(−k(k−1)/(2p)); for k>p the probability is zero.

With $k\approx c\sqrt{p}$, the exponent is approximately $-c^2/2$, so $\sqrt{p}$ is the typical collision scale. This is exact for the independent-uniform model; a random mapping's first-collision trajectory has the same product. A fixed polynomial $f(x)=x^2+c$ need not satisfy the model, so $\sqrt{p}$ is a heuristic rather than an unconditional worst-case or per-restart success guarantee. For balanced semiprimes $p\approx\sqrt{n}$ the heuristic is $n^{1/4}$ iterations; bit costs of modular operations and gcd are separate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Independence and Floyd’ s exact meeting bound

Repeating a fixed base does not create independent events: 2047 passes base 2 on every repetition. The per-round bounds multiply only when the legal bases are sampled independently as specified.

A trajectory in a finite function graph has tail length $\mu \geq 0$ and cycle length $\lambda \geq 1$. After k Floyd rounds the positions are x_k and x_{2k}. Choose k as the least multiple of λ at least $\max(\mu,1)$. Both positions lie on the cycle and their index difference k is divisible by λ, so they meet, with $k \leq \mu+\lambda$. This meeting bound is deterministic, but equality of positions need not produce a proper factor. An $O(\sqrt{p})$ estimate additionally assumes behavior resembling a random mapping.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A run that reaches gcd=n

Let n=15, $x_0=0$ and $f(x)=x^2+2 \bmod 15$. Round 1 gives slow=2, fast=6 and $\gcd(|2-6|,15)=1$. Round 2 gives slow=6 and fast=f(f(6))=f(8)=6, hence $\gcd(0,15)=15$. The two states meet modulo all of n; returning n is not a successful nontrivial factor.

Retry with a new seed or function, or report that this run found no factor when its work budget is exhausted. Failure does not prove n prime and 15 is not a nontrivial factorization of 15.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RSA key construction

Choose distinct primes p=11, q=13, n=143 and $\phi(n)=120$. Choose e=7 with gcd(7,120)=1; d=103 since $7 \cdot 103=721 \equiv 1 \pmod{120}$. Publish (n,e); keep d,p,q secret.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RSA encryption and decryption

Represent a message by $0 \le M < n$. For M=9, $C=9^7 \bmod 143=48$; decrypt $48^{103} \bmod 143=9$. These small numbers are only for teaching.

- p=11, q=13, n=143
- e=7, d=103
- 9 → 48 → 9
- encrypt: $9^7 \bmod 143 = 48$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RSA correctness

ed=1+kϕ(n). Modulo p, if p divides M both sides are 0; otherwise Fermat gives $M^{ed} \equiv M$. The same holds modulo q. CRT gives $M^{ed} \equiv M \pmod{n}$, even for non-coprime messages.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RSA also recovers non-units

Choose distinct primes p=11, q=17; n=187, ϕ(n)=160, e=7 and d=23, since 7 · 23=161≡1 (mod 160). The message m=11 has gcd(11,187)=11 but is a valid residue modulo n.

Encryption: $11^2 \equiv 121$ and $11^4 \equiv 55$, so $11^7 \equiv 55 \cdot 121 \cdot 11 \equiv 88 \pmod{187}$. Decryption: $88^2 \equiv 77$, $88^4 \equiv 132$, $88^8 \equiv 33$, $88^{16} \equiv 154$. Since 23=16+4+2+1, 154 · 132≡132, 132 · 77≡66 and 66 · 88≡11. Hence $88^{23} \equiv 11 \pmod{187}$.

Recover a non-unit message in each prime component

mod 11: message is 0 — m=0 —(7th power)→ c=0 —(23rd power)→ m′=0

mod 17: message is a unit — m=11 —(7th power)→ c=3 —(23rd power)→ m′=11

Both components recover the message; CRT gives 11 mod 187

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why RSA works for non-units

Let $ed \equiv 1 \pmod{(p-1)(q-1)}$. For each $r \in \{p,q\}$, if $r \mid m$ then $m^{ed} \equiv m \equiv 0 \pmod{r}$; otherwise Fermat's theorem and $r-1 \mid ed-1$ give $m^{ed} \equiv m \pmod{r}$. CRT then gives $m^{ed} \equiv m \pmod{pq}$.

In the example, modulo 11 the components are $11 \to 0$ and $88 \to 0$; modulo 17 they are $11 \to 11$ and $88 \to 3$, with $3^{23} \equiv 11$. CRT uniquely reconstructs 11 modulo 187. The proof assumes distinct primes p,q, positive e,d and the stated congruence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Malleability of textbook RSA

Unencoded $E(m)=m^e \bmod n$ is deterministic, and $E(m_1)E(m_2) \equiv E(m_1 m_2) \pmod{n}$. Given $c=E(m)$, anyone can choose $r \in U(n)$ and compute $c' = c \cdot r^e \bmod n$; it decrypts to $mr \bmod n$. This algebraic malleability does not require knowing m or the private key.

For $n=187$, $e=7$, $c=88$ and $r=2$, $2^7 \equiv 128$ and $c' \equiv 88 \cdot 128 \equiv 44$. Decrypting gives $44^{23} \equiv 22 = 2 \cdot 11$. This does not claim every real protocol reveals plaintext, but shows that the bare exponentiation is insufficient for the desired security; practical encryption requires standardized randomized encoding and an appropriate security model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A separate flaw in textbook RSA signatures

Suppose signing is $\sigma = m^d \bmod n$ and verification only checks $\sigma^e \equiv m \pmod n$. An attacker may first choose σ and then set $m = \sigma^e \bmod n$, producing a verifying message/signature pair without the private key. The attacker chooses σ and in general cannot also prescribe an arbitrary target message; the demonstrated claim is one valid unsigned pair under this message space and bare verification rule.

For $n = 55$ and $e = 3$ choose $\sigma = 12$. Then $m = 12^3 \bmod 55 = 23$, so $12^3 \equiv 23$ passes verification. This is a signature-verification flaw, distinct from the ciphertext malleability discussed earlier. Practical signatures require suitable hashing, encoding, and strict verification.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Scope and practice

This chapter gives mathematical algorithms. Practical RSA requires standardized encoding and randomized padding, secure parameters, and side-channel resistant implementation. The primality bound relies on the stated sampling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 31. MIT 6.046J Fall 2004 Lecture 7, p. 33, and Fall 2005 Lecture 8, p. 13: inverses modulo a prime.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study